



Laporan Praktikum Algoritma & Pemrograman
Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251235
Nama Lengkap	NATALIA GRECIA PUTRI
Minggu ke / Materi	14 / Fungsi Rekursif

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

Pengertian Rekursif

Fungsi rekursif adalah fungsi yang didalam tubuhnya memanggil dirinya sendiri. Dengan kata lain, sebuah fungsi dikatakan rekursif apabila dapat mendefinisikan prosesnya melalui pemanggilan terhadap fungsi yang sama, Fungsi rekursif biasanya digunakan untuk menyelesaikan permasalahan matematika yang memiliki pola berulang dan terstruktur. Meskipun demikian, penggunaan fungsi rekursif memerlukan perhatian khusus agar pemanggilan dapat berhenti pada kondisi tertentu dan tidak menghabiskan seluruh memori yang tersedia.

Apabila tidak dikendalikan dengan tepat, fungsi rekursif berpotensi berjalan tanpa henti layaknya perulangan tak terbatas (*infinite loop*), yang pada akhirnya dapat menyebabkan program berhenti merespons (*hang*). Oleh sebab itu, setiap fungsi rekursif wajib memiliki dua blok utama:

- **Base case:** kondisi yang memiliki titik berhenti dan proses rekursi. Ketika kondisi ini terpenuhi, fungsi tidak lagi memanggil dirinya dan proses dihentikan.
- **Recursive case:** bagian yang memiliki instruksi pemanggilan fungsi itu sendiri secara berulang, dan akan terus dieksekusi hingga akhirnya mencapai *base case*.

Kelebihan Rekursif

Fungsi rekursif memiliki sejumlah keunggulan yang membuatnya berguna dalam pemrograman, antara lain:

- Penulisan kode menjadi lebih ringkas dan elegan, sehingga struktur program terlihat lebih bersih dan mudah dibaca.

- Permasalahan yang kompleks dapat dipecah menjadi sub-masalahn yang lebih kecil, sehingga penyelesaiannya lebih tersruktur dan sistematis.

Namun disisi lain, fungsi rekursif juga memiliki beberapa kelemahan yang perlu diperhatikan:

- Konsumsi memori yang lebih besar, karena setiap pemanggilan fungsi membutuhkan alikasi ruang memori tambahan untuk menyimpan state-nya masing-masing.
- Kecepatan dan efisiensi dalam pengolahan data cenderung lebih rendah dibandingkan pendekatan iterasi.
- Proses debugging pada fungsi rekursif relatif lebihsulit dilakukan dan alur eksekusinya terkadang sulit untuk dipahami.

Bentuk Umum dan Studi Kasus

Secara umum struktur dasar dari sebuah fungsi rekursif dalam Python dapat dituliskan sebagai berikut:

```

1     def function_name(parameter_list):
2         ...
3         function_name(...)
4         ...

```

Pada dasarnya, setiap fungsi rekursif selalu memiliki padanan solusi iterasinya. Salah satu contoh kasus yang paling umum digunakan untuk memahami rekursi adalah perhitungan faktorial. Faktorial dari suatu bilangan n (ditulis n!) didefinisikan sebagai hasil perkalian beruntundari 1 x 2 x 3 x ... n.

Algoritma untuk mengitung faktorial secaraiteratif dapat dilakukan melalui langkah-langkah berikut:

- Minta nilai n dari pengguna.
- Siapkan variabel total untuk menampung hasil perkalian, dengan awal 1.
- Lakukan perulangan dari i = 1 hingga n, dengan operasi total = total * i pada setiap iterasi.
- Tampilkan nilai total sebagai hasil akhir.

Apabila pendekatan rekursif yang digunakan, perhitungan faktorial dapat dirumuskan secara matematis sebagai berikut:

$$\text{fact}(n) = \begin{cases} 1 & \text{if } n = 0 \\ n \cdot \text{fact}(n - 1) & \text{if } n > 0 \end{cases}$$

Berdasarkan rumus tersebut, pseudocode rekursif untuk fungsi faktorial dapat disusun sebagai berikut:

Pseudocode (recursive):

```
function factorial is:
input: integer n such that n >= 0
output: [n x (n-1) x (n-2) x ... x 1]

    1. if n is 0, return 1
    2. otherwise, return [ n x factorial(n-1) ]

end factorial
```

Tujuan Fungsi

- Fungsi ini bertujuan untuk menghitung nilai faktorial dari suatu bilangan bulat non-negatif.
- Nilai faktorial dilambangkan dengan $n!$, yang merupakan hasil perkalian berurutan dari n sampai 1, yakni $n! = n \times (n-1) \times (n-2) \times \dots \times 1$.

Input

- Fungsi menerima satu parameter berupa bilangan bulat n .
- Nilai n harus memenuhi syarat $n \geq 0$, karena faktorial tidak terdefinisi untuk bilangan negatif.

Output

- Fungsi menghasilkan nilai faktorial dari n sebagai keluaran akhirnya.

Logika dasar rekursif

1. Kasus Dasar (*Base Case*)

- Ketika nilai n sama dengan 0, fungsi langsung mengembalikan nilai 1.
- Ketentuan ini sesuai dengan aturan matematika bahwa $0! = 1$, sehingga berfungsi sebagai titik henti dari proses rekursif.

2. Kasus Rekursif (*Recursive Case*)

- Untuk nilai n yang lebih besar dari , fungsi akan menghasilkan nilai $n \times \text{faktorial}(n-1)$.
- Pada tahap ini fungsi memanggil dirinya sendiri dengan argumen yang satu nilai lebih kecil, membentuk rantai pemanggilan bertingkat hingga akhirnya mencapai base case.

Contoh Alur Eksekusi

Apabila fungsi dipanggil dengan `faktorial(3)`, maka rangkaian proses yang terjadi adalah sebagai berikut:

- `faktorial(3)` memanggil 3x `faktorial(2)`
- `faktorial(2)` memanggil 2x `faktorial(1)`
- `faktorial(1)` memanggil 1x `faktorial(0)`
- `faktorial(0)` mengembalikan nilai 1 sesuai base case
- Nilai kemudian dikalkulasi mundur: $1 * 1 * 2 * 3 = 6$

Keunggulan Menggunakan Rekursi

- Struktur kode yang dihasilkan lebih ringkas dan intuitif, karena mengikuti pola pemecahan masalah secara alami seperti yang ada dalam definisi matematisnya.
- Sangat sesuai diterapkan pada permasalahan yang bersifat berulang atau dapat dipecah menjadi sub-masalah sejenis, seperti perhitungan faktorial, deret Fibonacci, dan sejenisnya.

Berikut implementasi fungsi faktorial secara rekursif dalam Python:

```
def faktorial(n):
    if n==0 or n==1:
        return 1
    else:
        return faktorial(n-1) * n

print(faktorial(4))
```

Penjelasan dari kode di atas adalah sebagai berikut:

- Base case ditetapkan pada kondisi $n == 0$ atau $n == 1$, yang mengembalikan nilai 1. Ini sesuai dengan definisi matematika bahwa $0! = 1$ dan $1! = 1$.
- Pada recursive case, apabila nilai n lebih besar dari 1, fungsi memanggil dirinya sendiri dengan argumen $n - 1$, kemudian mengalikan hasilnya dengan n sebelum dikembalikan.
- Proses ini akan terus berlanjut secara bertingkat hingga pemanggilan berhasil mencapai base case.

Berikut adalah ilustrasi alur eksekusi fungsi faktorial secara rekursif:

```
1.
2. calc_factorial(4)           # 1st call with 4
3. 4 * calc_factorial(3)       # 2nd call with 3
4. 4 * 3 * calc_factorial(2)   # 3rd call with 2
5. 4 * 3 * 2 * calc_factorial(1) # 4th call with 1
6. 4 * 3 * 2 * 1               # return from 4th call as number=1
7. 4 * 3 * 2                   # return from 3rd call
8. 4 * 6                       # return from 2nd call
9. 24                          # return from 1st call
```

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

SOAL 1

https://github.com/grec12/71251235_nataliagrecia.git

```
windsun: Refactor | Explain | Generate Docstring | X
1  def prima(n, pembagi=2):
2      if n <= 1:
3          return False
4      if pembagi * pembagi > n:
5          return True
6      if n % pembagi == 0:
7          return False
8      return prima(n, pembagi + 1)
9
10 angka = int(input("Masukkan angka: "))
11 if prima(angka):
12     print(f"{angka} adalah bilangan prima.")
13 else:
14     print(f"{angka} bukan bilangan prima.")
```

```
Masukkan angka: 7
7 adalah bilangan prima.
```

Penjelasn kode:

- Fungsi `prima(n, pembagi=2)` didefinisikan dengan dua parameter: `n` sebagai bilangan yang akan diperiksa, dan `pembagi` dengan nilai awal 2.
- Kondisi pertama memeriksa apakah `n` kurang dari atau sama dengan 1. Jika iya, fungsi mengembalikan `False` karena bilangan prima harus berupa bilangan bulat positif yang lebih dari 1.
- Base case ditentukan dengan mengecek apakah kuadrat dari `pembagi` (`pembagi * pembagi`) sudah melampaui nilai `n`. Jika kondisi ini terpenuhi, artinya tidak ditemukan pembagi yang valid dan bilangan tersebut dinyatakan prima dengan mengembalikan `True`.
- Apabila `n` habis dibagi oleh `pembagi` (`n % pembagi == 0`), maka bilangan tersebut bukan prima dan fungsi mengembalikan `False`.

- Pada recursive case, fungsi memanggil dirinya sendiri dengan nilai pembagi + 1, untuk melanjutkan pemeriksaan ke pembagi berikutnya.
- Setelah fungsi didefinisikan, program meminta input dari pengguna dan menampilkan pesan yang menyatakan apakah bilangan tersebut prima atau bukan berdasarkan nilai yang dikembalikan fungsi.

SOAL 2

```

1  def palindrom(s):
2      s = ''.join(filter(str.isalnum, s)).lower()
3      if len(s) <= 1:
4          return True
5      if s[0] == s[-1]:
6          return palindrom(s[1:-1])
7      else:
8          return False
9
10  kalimat = input("Masukkan kalimat: ")
11  if palindrom(kalimat):
12      print("Kalimat tersebut adalah palindrom.")
13  else:
14      print("Kalimat tersebut bukan palindrom.")

```

Masukkan kalimat: kasur ini surak
Kalimat tersebut bukan palindrom.

Pejelasan kode:

- Sebelum dilakukan pemeriksaa, seluruh karakter dalam kalimat diubah mejadi huruf dan disusun sehingga hanya karkter alfanumerik (huruf dan angka) yang diperhatikan. Spasi dan tanda baca diabaikan.
- Base case ditetapkan ketika panjang kalimat kurang dari atau sama dengan 1, yang berarti kalimat hanya tersisa satu karakter atau kosong dan langsung dianggap palindrom dengan mengembalikan True.
- Pada setiap pemanggilan rekursif, karakter pertama dibandingkan dengan karakter terakhir dari kalimat. Jika keduanya tidak sama, fungsi mengembalikan False.

- Jika karakter pertama dan terakhir cocok, fungsi memanggil dirinya sendiri dengan kalimat yang telah dipotong dari kedua ujungnya (kalimat [1:-1]), untuk memeriksa bagian dalamnya.
- Proses ini berulang hingga seluruh pasangan karakter telah diperiksa atau ditemukan pasangan yang tidak cocok.
- Program kemudian menampilkan pesan yang menyatakan apakah kalimat yang dimasukkan merupakan palindrom atau tidak.

SOAL 3

```

1  def jumlah_deret_ganil(n):
2      if n <= 1:
3          return 1
4      else:
5          return n + jumlah_deret_ganil(n - 2)
6
7  try:
8      n = int(input("Masukkan nilai n: "))
9      if n < 1:
10         print("Nilai n harus lebih besar dari 0.")
11     else:
12         hasil = jumlah_deret_ganil(n)
13         print(f"Jumlah deret ganjil adalah {hasil}")
14 except ValueError:
15     print("Masukkan angka bulat positif!")

```

```

Masukkan angka: 7
7 adalah bilangan prima.

```

Penjelasan cara kerja program:

- Fungsi jumlah_deret_ganil(n) dirancang untuk menjumlahkan seluruh bilangan ganjil dari 1 hingga n.
- Base case ditetapkan ketika n kurang dari atau sama dengan 1, di mana fungsi langsung mengembalikan nilai 1 sebagai titik henti rekursi.
- Pada recursive case, apabila n lebih besar dari 1, fungsi mengembalikan n ditambah dengan hasil pemanggilan dirinya sendiri menggunakan argumenn 2. Pengurangan sebesar 2 memastikan bahwa hanya bilangan-bilangan ganjil yang dijumlahkan.

- Program menggunakan blok try-except untuk memvalidasi input pengguna. Jika nilai n kurang dari 1, program menampilkan pesan bahwa nilai harus lebih besar dari 0.
- Jika input bukan bilangan bulat, program menampilkan pesan kesalahan.
- Setelah validasi berhasil, program memanggil fungsi dan menampilkan hasil penjumlahan deret ganjil.

SOAL 4

```

1  def jumlah_digit(n):
2      if n < 10:
3          return n
4      else:
5          return n % 10 + jumlah_digit(n // 10)
6
7  try:
8      bilangan = int(input("Masukkan bilangan: "))
9      hasil = jumlah_digit(bilangan)
10     print(f"Jumah digit dari {bilangan} adalah {hasil}")
11 except ValueError:
12     print("Masukkan bilangan bulat yang valid.")

```

```

Masukkan bilangan: 234
Jumah digit dari 234 adalah 9

```

Penjelasan kode:

- Fungsi jumlah_digit(n) dirancang untuk menjumlahkan seluruh digit yang membentuk sebuah bilangan bulat.
- Base case ditetapkan ketika nilai n kurang dari 10, artinya bilangan hanya terdiri dari satu digit dan langsung dikembalikan sebagai hasilnya.
- Pada recursive case, apabila n memiliki lebih dari satu digit, fungsi mengambil digit terakhir menggunakan operasi modulus ($n \% 10$), kemudian menjumlahkannya dengan hasil pemanggilan rekursif terhadap sisa digit yang diperoleh melalui pembagian bilangan bulat ($n // 10$).
- Proses ini terus berulang hingga seluruh digit telah diproses dan dijumlahkan.
- Input pengguna divalidasi menggunakan blok try-except. Apabila input bukan bilangan bulat yang valid, program menampilkan pesan kesalahan yang sesuai.

SOAL 5

```
1 def kombinasi(n, s):
2     if s == 0 or s == n:
3         return 1
4     else:
5         return kombinasi(n - 1, s - 1) + kombinasi(n - 1, s)
6
7 v = int(input("Masukkan nilai v: "))
8 t = int(input("Masukkan nilai t: "))
9 hasil = kombinasi(v, t)
10 print(f"Hasil Kombinasinya adalah {hasil}")
```

```
Masukkan nilai v: 7
Masukkan nilai t: 5
Hasil Kombinasinya adalah 21
```

Penjelasan cara kerja program:

- Fungsi kombinasi(n, k) mengimplementasikan rumus kombinasi secara rekursif mengikuti prinsip segitiga Pascal.
- Base case ditetapkan pada dua kondisi: ketika k sama dengan 0, atau ketika k sama dengan n. Pada kedua kondisi ini, nilai kombinasi pasti 1 dan langsung dikembalikan.
- Pada recursive case, fungsi mengembalikan hasil penjumlahan dari dua pemanggilan rekursif: kombinasi(n-1, k-1) dan kombinasi(n-1, k). Pola ini sesuai dengan sifat segitiga Pascal, di mana setiap nilai merupakan jumlah dari dua nilai di atasnya.
- Program meminta pengguna untuk memasukkan nilai n dan k, kemudian memanggil fungsi kombinasi dan menampilkan hasilnya.